SPARROW: Scalable Taxonomy Induction via Structure-Preserving Partitioning and Constraint-Guided Merging

Yirui Zhang **Yixuan Tang*** **Yandong Sun** **Mong-Li Lee** **Anthony Kum Hoe Tung**
School of Computing, National University of Singapore
{yirui_z, yixuan, yandong, leeml, atung}@comp.nus.edu.sg

Abstract

Taxonomy induction aims to organize concept sets into coherent hierarchical structures. Recent LLM-based methods can induce taxonomies directly from flat term lists, avoiding the need for corpora, but degrade sharply as concept sets scale up. We argue that this degradation stems not only from context length limitations, but also from structural failures in hierarchical reasoning. To address this, we adopt a divide-and-merge paradigm that partitions concepts into smaller subsets, induces local taxonomies, and merges them into a global hierarchy. However, we identify two structural failure modes inherent to this paradigm: **Structural Fragmentation**, where partitioning weakens local hierarchical signals, and **Parent Displacement**, where locally plausible relations are misplaced in the global hierarchy. To address both, we propose **SPARROW**, a scalable taxonomy induction framework that combines structure-preserving spectral partitioning to retain hierarchical connectivity within each block, and constraint-guided incremental fusion that treats block-level relations as structural constraints rather than ground truth for global placement. Experiments on large-scale benchmarks show that SPARROW consistently achieves the strongest global structural quality, as measured by Ancestor F1, across backbones. The code is available at https://github.com/rebeccazyr/SPARROW.

1 Introduction

Taxonomies are fundamental structures for organizing concepts and supporting downstream applications such as semantic search (Kang et al., 2024), recommendation (Tan et al., 2022), question answering (Sun et al., 2026a,b), and, more broadly, knowledge integration (Tung, 2026). Taxonomy induction has therefore been widely studied, with approaches ranging from pattern-based (Roller et al., 2018) and topic model-based methods (Blei et al., 2010) to embedding-based techniques (Zhang et al., 2018; Shang et al., 2020), all of which rely on large text corpora to surface hierarchical signals. Recent work has shown that LLMs, by encoding rich world knowledge during pretraining, can infer hierarchical relations directly from term names (Chen et al., 2023; Zeng et al., 2024), achieving strong performance on small to medium-sized concept sets without corpus dependency. However, their performance deteriorates sharply as the concept set scales up. Zeng et al. (2024) show that even state-of-the-art LLM-based methods lose the ability to faithfully cover all input concepts and exhibit consistent structural degradation as the term set grows, revealing fundamental limitations in both context handling capacity and hierarchical reasoning ability. More fundamentally, we argue that this degradation reflects an inherent difficulty of LLM-based approaches: maintaining globally coherent hierarchical relations under localized reasoning becomes increasingly difficult as scale grows, even when individual placements appear locally correct.

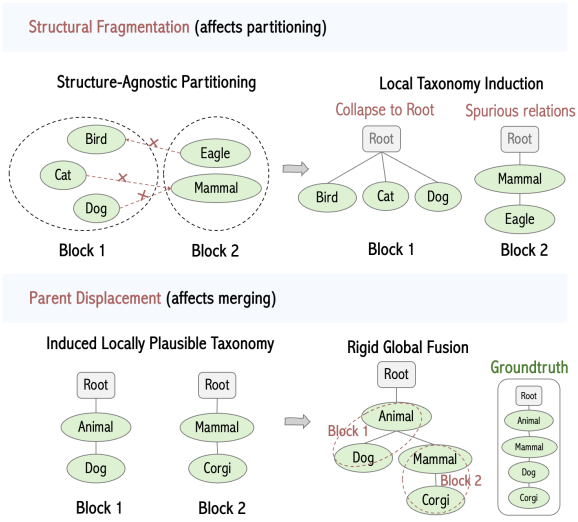


Figure 1: Two structural failure modes.

*Corresponding author.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 34320–34332
October 24-29, 2026 ©2026 Association for Computational Linguistics

A natural response to this limitation is to adopt a divide-and-merge strategy: partition the concept set into smaller subsets, induce local taxonomies independently, and merge the resulting structures into a global hierarchy. However, a naive application of this paradigm can be brittle at scale because it overlooks two structural failure modes, illustrated in Figure 1.

Structural Fragmentation. Partitioning inevitably severs cross-subset parent-child relations, but the critical failure mode lies in partition quality. When concepts within a block share insufficient latent hierarchical structure, local induction tends to collapse concepts onto the root, offloading all structural recovery to the merge stage, or fabricates spurious relations that persist irrecoverably through the pipeline. Effective partitioning should therefore group concepts likely to share genuine hierarchical relations, maximizing local inductive signal despite the true taxonomy being unknown at partition time.

Parent Displacement. Even when block taxonomies are locally correct, block-level parent-child relations do not necessarily hold globally. Because a block contains only a subset of concepts, intermediate nodes absent from the block cause ancestor-descendant relations to compress into spurious direct edges. Naively lifting block taxonomies as rigid units therefore embeds these structural errors irrecoverably into the global hierarchy. Block-level relations should instead constrain rather than determine global placement, with ambiguous placements resolved through explicit reasoning over its candidate-specific hierarchical context.

To address these failure modes, we propose **SPARROW** (Structure-Preserving pARtitioning and constRaint-guided meRging for taxonOmies), a scalable taxonomy induction framework that follows a divide-and-merge paradigm. In the divide stage, structure-preserving partitioning applies spectral clustering to preserve hierarchical connectivity within each block, retaining latent parent-child signal for reliable local induction under bounded context. In the merge stage, constraint-guided incremental fusion treats block-level relations as structural constraints to scope candidate parents rather than lifting them as ground truth, with final placement resolved through explicit LLM reasoning.

We evaluate SPARROW on multiple taxonomy induction benchmarks with concept sets ranging from tens to ten thousand nodes, using both strong-context and limited-context backbone models. Experimental results show that SPARROW consistently achieves greater structural stability than existing methods, with particularly strong improvements as scale increases.

Our main contributions are:

- We identify two structural failure modes in **large-scale LLM-based taxonomy induction**: **Structural Fragmentation**, where partitioning weakens local hierarchical signals, and **Parent Displacement**, where locally plausible relations lead to incorrect global ancestor placement.
- We propose **SPARROW**, a scalable divide-and-merge framework that combines **structure-preserving partitioning** with **constraint-guided incremental fusion**, treating block-level relations as structural constraints rather than fixed global decisions.
- We demonstrate that SPARROW achieves **stronger global structural consistency and scalability** across multiple taxonomy benchmarks, with consistently higher ancestor-level accuracy under both strong and limited-context LLM backbones.

2 Related Work

2.1 Corpus-based Taxonomy Induction

Taxonomy induction from text corpora exploits distributional evidence to approximate hypernym–hyponym relations. Early approaches rely on lexical-syntactic patterns (Roller et al., 2018; Snow et al., 2004) or probabilistic topic models such as hLDA (Blei et al., 2010, 2003), while clustering-based methods construct hierarchies over learned term embeddings (Zhang et al., 2018; Shang et al., 2020). More recent work incorporates LLMs into corpus-based pipelines to improve taxonomy quality (Kargupta et al., 2025; Lahiri et al., 2025; Zhu et al., 2025; Tang et al., 2026). However, these methods assume access to document corpora and aim to induce taxonomies from distributional evidence, where the concept inventory is not predefined. In contrast, our setting assumes a fixed flat concept set without accompanying text, and focuses on inferring the hierarchical structure among the given concepts.

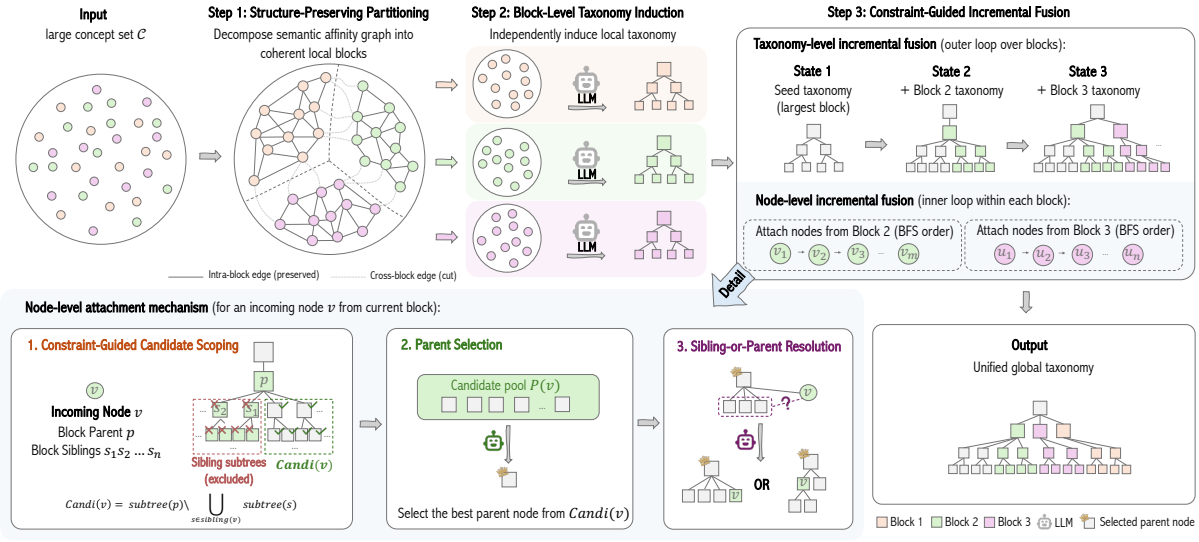


Figure 2: Overview of **SPARROW**, a divide-and-merge framework for large-scale taxonomy induction. SPARROW first partitions the input concepts into semantically coherent blocks through *Structure-Preserving Partitioning*, and induces local taxonomies through *Block-Level Taxonomy Induction*. The resulting block-level taxonomies are then fused into a unified global taxonomy through *Constraint-Guided Incremental Fusion*, which consists of an outer loop that sequentially inserts local taxonomies into the evolving global structure, and an inner loop that attaches incoming nodes under structural constraints.

2.2 Term-based Taxonomy Induction

The emergence of large language models has enabled taxonomy induction directly from flat concept sets by leveraging strong semantic abstraction capabilities. Prior work explores prompting and fine-tuning strategies for hierarchical construction (Chen et al., 2023), and layer-wise prompting schemes to improve structural consistency (Zeng et al., 2024). However, these approaches require all candidate terms to be present within a single inference context, whether generating the taxonomy in one pass or layer by layer. Such designs are inherently limited by context length constraints, making it difficult to maintain hierarchical consistency as the concept set grows.

2.3 Taxonomy Merging and Structural Fusion

Taxonomy merging integrates multiple hierarchical structures into a unified taxonomy (Cheng and Xia, 2023). Existing methods based on semantic node matching (Raunich and Rahm, 2014), logic-based consistency enforcement (Chen et al., 2014), and heuristic conflict resolution (Chen et al., 2022; Babalou and König-Ries, 2023) all presuppose that input taxonomies share overlapping nodes or explicit cross-taxonomy correspondences to anchor the integration. This condition does not hold when merging independently induced partial hierarchies with disjoint vocabularies.

Taxonomy expansion and completion methods (Yu et al., 2020; Xia et al., 2024; Mishra et al., 2025; Wang et al., 2021) insert individual concepts into a fixed backbone by selecting appropriate parent or child nodes. In contrast, our fusion procedure attaches structured block-level subtrees onto an evolving, incomplete backbone, where the internal relations within each subtree constrain attachment decisions and the backbone itself grows incrementally throughout the process.

3 Problem Formulation

We study the single-parent setting, where each non-root concept has exactly one parent, and represent a taxonomy as a rooted tree $\mathcal{T} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes a set of concepts and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes *is-a* relations. Given a flat concept set $\mathcal{C} = \{c_1, \dots, c_N\}$ as input, taxonomy induction aims to infer a valid edge set $\mathcal{E}$ such that $(\mathcal{C}, \mathcal{E})$ forms a valid rooted tree. We focus on the large-scale setting where $|\mathcal{C}|$ is too large for joint inference.

4 Methodology

4.1 Overview

Given a large concept set $\mathcal{C}$, SPARROW constructs a globally coherent taxonomy by explicitly decoupling *local semantic abstraction* from *global struc-*

tural reasoning. The framework follows a divide-and-merge paradigm, operationalized through three tightly coupled stages: (1) *structure-preserving partitioning*, which decomposes $\mathcal{C}$ into locally coherent blocks; (2) *block-level taxonomy induction*, which independently induces a local taxonomy within each block under bounded context; and (3) *constraint-guided incremental fusion*, which integrates the resulting block-level taxonomies into a unified global hierarchy under structural constraints. Figure 2 provides an overview of the complete SPARROW pipeline. This design directly targets the two structural failure modes identified earlier: *Structural Fragmentation* during partitioning and *Parent Displacement* during merging.

4.2 Structure-Preserving Partitioning

The *Structure-Preserving Partitioning* stage decomposes the concept set $\mathcal{C}$ into subsets of optimal size within LLM context budgets, while preserving coherent local sub-hierarchies. Gold subtrees typically form connected neighborhoods in the embedding kNN graph, yet are only moderately concentrated around a shared centroid (Appendix A.5). An effective partitioning strategy should therefore preserve local connectivity rather than optimize geometric compactness alone. This motivates spectral clustering, which can retain connected semantic structure within each block.

Concretely, each concept $c_i \in \mathcal{C}$ is encoded into dense semantic embeddings $\{e_i\}_{i=1}^{n}$. A sparse affinity graph is constructed over the top-k nearest neighbors of each concept. For a retained edge (i, j), cosine distance is $d_{ij} = 1 - \frac{e_i \cdot e_j}{\|e_i\|\|e_j\|}$ and its weight is $\exp(-\gamma d_{ij}^2)$, with $\gamma = 1.0$. Normalized spectral clustering (von Luxburg, 2007) is applied over this graph, with the block count B selected adaptively via the eigengap heuristic within a candidate range derived from concept set size. Concepts are partitioned into B disjoint semantic blocks $\{\mathcal{C}^{(1)}, \ldots, \mathcal{C}^{(B)}\}$, each serving as an independent bounded-context unit for local taxonomy induction.

4.3 Block-Level Taxonomy Induction

Given a block $\mathcal{C}^{(b)}$, SPARROW independently induces a local taxonomy by prompting an LLM to assign each concept a parent from within-block candidates. If no suitable parent can be identified, the concept is attached to the block root, deferring its global placement to the fusion stage. The output is a set of locally coherent block-level taxonomies that capture fine-grained semantic structure.

4.4 Constraint-Guided Incremental Fusion

Incremental Fusion. After block-level induction, SPARROW integrates the independently induced local taxonomies into a unified global hierarchy incrementally. The largest block is selected as the structural seed, maximizing initial hierarchical coverage. Remaining blocks are integrated one at a time into the evolving partial hierarchy in descending order of block size, with nodes within each block processed in BFS order so that a node's block-level parent is always placed before its descendants, enabling constraint-guided candidate scoping during attachment. Each attachment involves three sequential steps: constructing a constraint-guided candidate parent pool, selecting a valid parent, and resolving the node's structural role relative to existing children. Fusion preserves within-block ancestral relations while allowing cross-block attachments to refine direct edges.

Constraint-Guided Candidate Scoping. For each incoming node v to be attached to the evolving taxonomy, a constraint-guided candidate parent pool is constructed from the currently fused taxonomy. The construction directly exploits the structural information established during block-level induction: since v's parent-child and sibling relations within its block have already been determined, candidates are restricted to the subtree rooted at v's block-level parent p, excluding the subtrees of v's siblings whose parent-child relations with v have already been ruled out,

$$\mathrm{Candi}(v) = \mathrm{subtree}(p) \setminus \bigcup_{s \in sibling(v)} \mathrm{subtree}(s)$$

For block-root nodes with no block-level parent, $\mathrm{Candi}(v) = \mathcal{T}$, the set of all currently placed nodes. This scoping preserves consistency with block-level induction, reuses its semantic reasoning, and reduces the candidate pool to a structurally relevant subset, improving both precision and efficiency of the attachment decision.

Parent Selection. Given the constraint-guided candidate pool $\mathrm{Candi}(v)$, the top-K candidates are first retrieved by embedding similarity and passed to an LLM for final parent selection. Each candidate is provided with its full taxonomy path and ex-

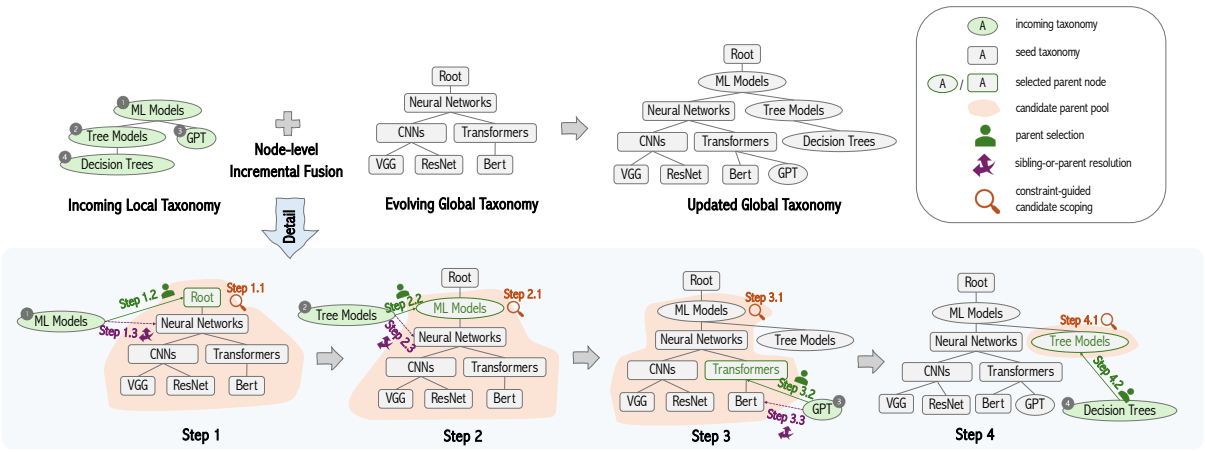


Figure 3: Running example of the *Node-level Incremental Fusion* mechanism in Figure 2. An incoming local taxonomy is integrated into the evolving global taxonomy through sequential node-level attachment. Each incoming node undergoes three stages: *Constraint-Guided Candidate Scoping*, *Parent Selection*, and *Sibling-or-Parent Resolution*, progressively updating the global taxonomy in BFS order.

isting children, supplying the structural context necessary for coherent placement (see Appendix A.7 for the full prompt). The global root is always included as a candidate, allowing the LLM to defer placement when the subsumption check finds no match. Repositioning is triggered incrementally during sibling-or-parent resolution: when a later node subsumes a deferred root child, that child is moved beneath the newly inserted node rather than being handled by a separate post-processing pass.

Sibling-or-Parent Resolution. Once a parent P is selected, a second LLM decision determines the structural role of v relative to the existing children of P. If v's semantic scope is comparable to that of the existing children, v is attached as a sibling; if v semantically subsumes any of those children, v is inserted as an intermediate node above the subsumed children. (Prompts in Appendix A.7).

Figure 3 illustrates a complete fusion example, showing how constraint-guided candidate scoping, parent selection, and sibling-or-parent resolution interact across four sequential node attachments when integrating an incoming local taxonomy into the evolving global taxonomy.

5 Experiments

5.1 Experimental Setup

Datasets. We conduct our main experiments on two taxonomies from distinct domains:

ACM Computing Classification System (CCS) (ACM, 2012), an expert-curated computer science taxonomy organized into six hierarchical levels, pruned to a strict tree of 1,768 concepts via single-parent filtering.

Google Product Taxonomy (Google) (Google, 2021), a large-scale taxonomy for e-commerce categorization comprising 5,595 product concepts across up to seven levels of depth.

Both datasets provide ground-truth hierarchical structures for quantitative evaluation, covering complementary domains of academic computing and e-commerce. To further evaluate cross-domain generalization and scalability, we additionally use SemEval TExEval-2 Food (Bordea et al., 2016) and the biomedical Medical Subject Headings (MeSH) (Lipscomb, 2000), including a 10K-concept setting; full results for both are reported in Appendix A.3. Sampling procedures and further details are provided in Appendix A.1.

Evaluation Metrics. We evaluate taxonomy quality using **Node F1**, **Edge F1**, and **Ancestor F1**.

Node F1 evaluates concept coverage. A node is considered correct if it appears in the induced taxonomy, regardless of its structural position. This metric reflects whether the model successfully recovers the set of target concepts.

Edge F1 evaluates local hierarchical correctness by comparing predicted parent–child relations against the gold taxonomy. Each directed edge is treated as an evaluation unit. This metric captures the accuracy of immediate parent assignments.

Ancestor F1 evaluates global hierarchical consistency. For each node, we consider its full ances-

Dataset	Method	Backbone	Node			Edge			Ancestor		
			P	R	F1	P	R	F1	P	R	F1
CCS	TaxoGPT	GPT-5	0.984	0.756	0.855	0.325	0.242	0.278	0.696	0.277	0.397
	Chain-of-Layer		1.000	0.519	0.683	0.600	0.301	0.401	0.751	0.165	0.271
	LLMscorer		1.000	1.000	**1.000**	0.490	0.474	**0.482**	0.488	0.464	0.476
	SPARROW		1.000	0.939	0.969	0.425	0.399	0.412	0.670	0.582	**0.623**
	TaxoGPT	LLaMA3-8B	0.262	0.016	0.030	0.050	0.003	0.006	0.029	0.001	0.002
	Chain-of-Layer		–	–	–	–	–	–	–	–	–
	LLMscorer		1.000	0.999	**1.000**	0.248	0.207	**0.226**	0.222	0.109	0.146
	SPARROW		0.972	0.741	0.841	0.113	0.087	0.100	0.560	0.315	**0.403**
Google	TaxoGPT	GPT-5	1.000	0.812	0.896	0.819	0.432	0.566	0.700	0.556	0.619
	Chain-of-Layer		1.000	0.438	0.609	0.636	0.274	0.383	0.893	0.246	0.386
	LLMscorer		1.000	0.991	**0.995**	0.597	0.580	**0.588**	0.402	0.519	0.453
	SPARROW		1.000	0.955	0.977	0.592	0.566	0.579	0.770	0.721	**0.745**
	TaxoGPT	LLaMA3-8B	0.944	0.017	0.033	0.125	0.002	0.004	0.171	0.002	0.004
	Chain-of-Layer		–	–	–	–	–	–	–	–	–
	LLMscorer		1.000	0.845	**0.916**	0.310	0.214	**0.253**	0.304	0.119	0.171
	SPARROW		0.923	0.500	0.649	0.282	0.153	0.198	0.247	0.257	**0.253**

Table 1: Performance comparison under 1K–scale taxonomy induction settings. "–" indicates failure to produce a valid taxonomy output due to context limitations.

tor set defined by the path from the node to the root. Predicted ancestor–descendant pairs are compared against the corresponding gold ancestor sets. This metric is particularly sensitive to errors in global positioning that Edge F1 alone cannot detect.

Baselines. We compare **SPARROW** against three LLM-based taxonomy induction baselines spanning one-pass generation, layer-wise construction, and pairwise scoring.

- **TaxoGPT** (Chen et al., 2023) formulates taxonomy induction as a text generation task. Given a set of concepts, the model is prompted in a few-shot setting to generate parent–child relations as textual statements.
- **Chain-of-Layer (CoL)** (Zeng et al., 2024) constructs taxonomies in a top-down, layer-wise manner. Starting from a root concept, the model iteratively expands the taxonomy by attaching subsets of entities at each level.
- **LLMscorer** adapts the parent-ranking formulation of LMScorer (Jain and Anke, 2022). It uses the backbone model to assess parent–child compatibility within retrieved candidate sets and constructs the taxonomy from the resulting parent assignments. Implementation details are provided in Appendix A.2.

5.2 Main Results

Table 1 presents taxonomy induction results on CCS and Google, each evaluated on a subset of 1,000 nodes, under both GPT-5[1] and LLaMA3-8B-Instruct[2] backbones.

Results under GPT-5. Even with GPT-5, large-scale taxonomy induction remains structurally challenging. SPARROW achieves the strongest global structural quality on both datasets, exceeding TaxoGPT in Ancestor F1 by 0.226 on CCS and 0.126 on Google. LLMscorer attains higher Node and Edge F1, consistent with its independent per-concept parent-selection design, which favors node coverage and locally accurate attachments. SPARROW nevertheless outperforms LLMscorer in Ancestor F1 by 0.147 on CCS and 0.292 on Google. Notably, on Google, this substantial global advantage is achieved with nearly identical Edge F1 (0.579 vs. 0.588). These results demonstrate that SPARROW achieves substantially stronger global hierarchical coherence through structure-preserving partitioning and constraint-guided fusion.

Results under LLaMA3-8B-Instruct. The limited-context setting is substantially more challenging. TaxoGPT suffers severe recall collapse on both datasets, while Chain-of-Layer cannot complete inference because its layer-wise prompts exceed the model's context window. SPARROW nevertheless achieves the highest Ancestor F1 on

[1] https://developers.openai.com/api/docs/models/gpt-5

[2] https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

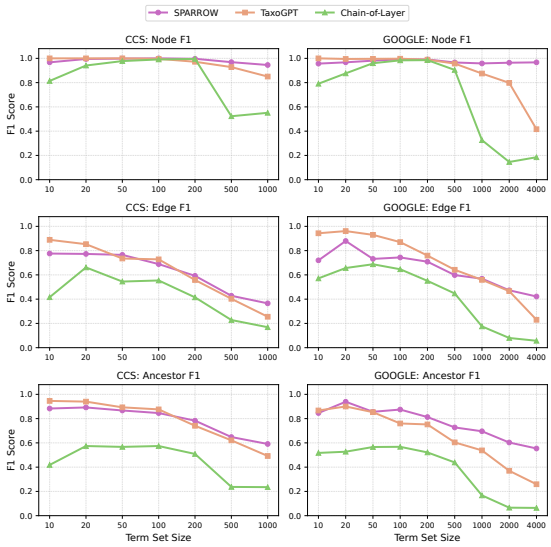


Figure 4: Scalability results on CCS and Google.

both datasets. Relative to TaxoGPT, it achieves approximately 28× higher Node F1 and over 200× higher Ancestor F1 on CCS, as well as over 60× higher Ancestor F1 on Google. Against LLMscorer, SPARROW achieves higher Ancestor F1 on CCS (0.403 vs. 0.146) and Google (0.253 vs. 0.171), despite lower Node and Edge F1. Notably, SPARROW with LLaMA3-8B-Instruct matches TaxoGPT with GPT-5 on CCS Ancestor F1 (0.403 vs. 0.397), despite using a substantially weaker backbone. This cross-backbone comparison further highlights the robustness of SPARROW's divide-and-merge framework under constrained backbone capacity and context.

5.3 Scalability Analysis

Figure 4 compares TaxoGPT, Chain-of-Layer, and SPARROW across increasing input scales. As the input size increases, both baselines degrade markedly, whereas SPARROW remains substantially more stable. The resulting performance gap is particularly pronounced in the 4K-node Google setting.

LLMscorer is not included in these curves because, under its per-concept inference protocol, a single sweep over all CCS and Google scales is estimated to consume approximately 4M tokens. Its 1K performance and measured token consumption are reported in Tables 1 and 5, respectively.

Cross-domain and extreme-scale evaluation. We further evaluate SPARROW on SemEval-Food and MeSH (Appendix A.3). SPARROW achieves the highest Node and Ancestor F1 on SemEval-Food and an Ancestor F1 of 0.487 in the 10K-node MeSH setting, compared with 0.011 for TaxoGPT and 0.015 for Chain-of-Layer. These results demonstrate cross-domain generalization and extend our scalability evaluation to 10K concepts.

Method	Overlap	IER	BID
K-means	0	0.620	0.133
K-means	$\tau = 0.60$	**0.902**	0.116
K-means	$\tau = 0.90$	0.798	0.122
Hierarchical	0	0.643	0.138
Spectral	0	0.766	**0.149**
Spectral	$\text{thr} = 0.25$	0.839	0.119
Spectral	$\text{thr} = 0.05$	0.773	0.137

Table 2: Comparison of clustering strategies on CCS at the 1K scale. IER and BID evaluate structure preservation in the divide stage.

5.4 Structure Preservation in the Divide Stage

An effective partitioning strategy should preserve as many ground-truth parent–child relations as possible within blocks while keeping the blocks compact. We quantify this with two complementary metrics.

Intra-block Edge Retention (IER) measures the fraction of ground-truth edges retained within the blocks:

$$\text{IER} = \frac{\left|\bigcup_{B \in \mathcal{B}} E^B_{\text{intra}}\right|}{|E_{\text{gt}}|}.$$

Block-normalized Intra-edge Density (BID) captures structural compactness, penalizing partitions that preserve edges only by forming excessively large blocks:

$$\text{BID} = \frac{\sum_{B \in \mathcal{B}} |E^B_{\text{intra}}|}{\sum_{B \in \mathcal{B}} N_B \log N_B},$$

where N_B is the number of nodes in block B.

As shown in Table 2, overlapping K-means with $\tau = 0.60$ achieves the highest IER. However, its lower BID suggests that this improvement relies on large, redundant blocks, weakening the scalability benefits of partitioning. In contrast, spectral clustering without overlap achieves the highest BID, producing more compact and structurally efficient partitions. We therefore adopt this method as our partitioning strategy. Appendix A.4 reports robustness to partitioning noise and sensitivity to block size.

Dataset	Split	Merge	Node			Edge			Ancestor		
			P	R	F1	P	R	F1	P	R	F1
CCS	✗	✗	0.984	0.756	0.855	0.325	0.242	0.278	0.696	0.277	0.397
	✓	✗	1.000	0.939	0.969	0.429	0.394	0.411	0.601	0.379	0.465
	✓	✓	1.000	0.939	**0.969**	0.425	0.399	**0.412**	0.670	0.582	**0.623**
Google	✗	✗	1.000	0.812	0.896	0.819	0.432	0.566	0.700	0.556	0.619
	✓	✗	1.000	0.955	0.977	0.585	0.543	0.563	0.714	0.499	0.587
	✓	✓	1.000	0.955	**0.977**	0.592	0.566	**0.579**	0.770	0.721	**0.745**

Table 3: Ablation study about SPARROW stages on CCS and Google.

Variant	Component	Node F1	Edge F1	Anc. F1
w/o Scope	Step 1	0.977	0.547	0.484
Sim.-only	Step 2	0.977	0.406	0.374
Node-only	Step 2	0.977	0.559	0.530
Always-sib.	Step 3	0.977	0.556	0.504
SPARROW	Full	0.977	**0.579**	**0.744**

Table 4: Merge-stage ablation study on Google at the 1K scale. "w/o Scope" removes constraint-guided candidate scoping (Step 1); "Sim.-only" and "Node-only" simplify parent selection (Step 2); "Always-sib." disables sibling-or-parent resolution (Step 3).

Method	Local	Merge	Total
TaxoGPT	9,813	–	9,813
Chain-of-Layer	62,038	–	62,038
LLMscorer	407,518	–	407,518
SPARROW	10,254	52,058	62,312

Table 5: API token consumption across different taxonomy induction methods. For SPARROW, we further decompose the cost into local induction and merge stages.

5.5 Ablation Study

We conduct an ablation study on CCS and Google at the 1K scale to analyze the contributions of the divide and merge stages in SPARROW as shown in Table 3. Removing both stages substantially reduces Edge F1 and Ancestor F1, indicating the difficulty of inducing a coherent hierarchy over a large concept set in a single pass. Introducing the divide stage significantly improves Edge F1 by decomposing the taxonomy into semantically coherent local subgraphs, reducing the input scale and allowing the model to induce parent-child relationships with greater precision within each block. However, Ancestor F1 remains limited, as locally correct subgraphs still lack global positional grounding across blocks. The full SPARROW framework achieves the strongest Ancestor F1 on both datasets, confirming that the divide stage improves local structural fidelity while the merge stage is indispensable for recovering globally coherent hierarchies.

Table 4 further validates the internal design of the merge stage on Google. Without constraint-guided candidate scoping, each node must consider all previously merged nodes as possible parents. This reduces Ancestor F1 by 0.260 relative to SPARROW and uses roughly 6.7 $\times$ more tokens, effectively negating the efficiency gains of the divide stage. For parent selection, Sim.-only and Node-only drop Ancestor F1 by 0.370 and 0.214 respectively compared to SPARROW, confirming that combining embedding-based filtering with path-aware LLM inference is critical for accurate parent resolution. Finally, Always-sib instantiates the *parent displacement* failure mode. By defaulting every attachment to a sibling relation, it restricts attachment points to block root nodes and degenerates into the rigid global fusion shown in Figure 1. This reduces Ancestor F1 by 0.240 relative to SPARROW. Together, these results demonstrate that all three components, constraint-guided candidate scoping, path-aware parent selection, and sibling-or-parent resolution, are individually necessary for the merge stage to recover globally coherent hierarchies.

5.6 Cost Analysis

Table 5 reports GPT-5 API token consumption on CCS at the 1K scale. TaxoGPT and SPARROW's local stage both rely on one-pass generation and have comparable token usage (9,813 vs. 10,254). Including fusion, SPARROW's total remains nearly identical to Chain-of-Layer (62,312 vs. 62,038 tokens), but its Ancestor F1 is substantially higher (0.623 vs. 0.271). Our LLMscorer adaptation replaces the original LMScorer's exhaustive $O(n^2)$ pair scoring with $O(n)$ LLM parent-selection queries over retrieved candidate sets. Even with this reduction, LLMscorer consumes 407,518 tokens, approximately $6.5\times$ as many as SPARROW, and achieves lower Ancestor F1 (0.476 vs. 0.623). These comparisons show

that SPARROW directs computation toward global reconciliation, yielding a stronger accuracy–cost trade-off than methods with comparable or greater token usage.

6 Conclusion

We present SPARROW, a scalable divide-and-merge framework for large-scale taxonomy induction that explicitly addresses two fundamental structural failure modes: structural fragmentation during partitioning and parent displacement during merging. Through structure-preserving spectral partitioning and constraint-guided incremental fusion, SPARROW decomposes large-scale induction into tractable local subproblems while preserving global hierarchical consistency. Experiments on CCS and Google across both strong- and limited-context backbones demonstrate that SPARROW consistently achieves superior ancestor-level accuracy and structural stability, with gains that become more pronounced at scale. These results suggest that the scalability bottleneck in taxonomy induction is fundamentally structural rather than model-specific, and that principled pipeline design can unlock strong hierarchical reasoning even from limited-capacity backbones.

Limitations

While SPARROW achieves strong performance, several limitations remain. SPARROW studies single-parent rooted trees and does not currently support polyhierarchical taxonomies represented as DAGs, in which concepts may have multiple parents. The method also assumes a complete input concept set, whereas important intermediate concepts may be missing in practice, potentially introducing structural bias. Finally, the iterative merge stage incurs token overhead, and improving its efficiency remains future work.

Acknowledgments

This research is supported by the Ministry of Education, Singapore, under its MOE AcRF TIER 1 Grant (T1 251RES2517).

References

ACM. 2012. ACM computing classification system. https://dl.acm.org/ccs.

Samira Babalou and Birgitta König-Ries. 2023. Towards building knowledge by merging multiple ontologies with comerger: A partitioning-based approach. *Appl. Ontology*, 18(4):307–341.

David M. Blei, Thomas L. Griffiths, and Michael I. Jordan. 2010. The nested chinese restaurant process and bayesian nonparametric inference of topic hierarchies. *J. ACM*, 57(2):7:1–7:30.

David M. Blei, Andrew Y. Ng, and Michael I. Jordan. 2003. Latent dirichlet allocation. *J. Mach. Learn. Res.*, 3:993–1022.

Georgeta Bordea, Els Lefever, and Paul Buitelaar. 2016. Semeval-2016 task 13: Taxonomy extraction evaluation (texeval-2). In *Proceedings of the 10th International Workshop on Semantic Evaluation, SemEval@NAACL-HLT 2016, San Diego, CA, USA, June 16-17, 2016*, pages 1081–1091. The Association for Computer Linguistics.

Boqi Chen, Fandi Yi, and Dániel Varró. 2023. Prompting or fine-tuning? A comparative study of large language models for taxonomy construction. In *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS 2023 Companion, Västerås, Sweden, October 1-6, 2023*, pages 588–596. IEEE.

Mao Chen, Chao Wu, Zongkai Yang, Sanya Liu, Zengzhao Chen, and Xiuling He. 2022. A multi-strategy approach for the merging of multiple taxonomies. *J. Inf. Sci.*, 48(3):283–303.

Mingmin Chen, Shizhuo Yu, Nico M. Franz, Shawn Bowers, and Bertram Ludäscher. 2014. Euler/x: A toolkit for logic-based taxonomy integration. *CoRR*, abs/1402.1992.

Yi-Yun Cheng and Yilin Xia. 2023. A systematic review of methods for aligning, mapping, merging taxonomies in information sciences. *J. Documentation*, 79(6):1413–1439.

Google. 2021. Google product taxonomy. https://www.google.com/basepages/producttype/taxonomy.en-US.txt.

Devansh Jain and Luis Espinosa Anke. 2022. Distilling hypernymy relations from language models: On the effectiveness of zero-shot taxonomy induction. In *Proceedings of the 11th Joint Conference on Lexical and Computational Semantics, *SEM@NAACL-HLT 2022, Seattle, WA, USA, July 14-15, 2022*, pages 151–156. Association for Computational Linguistics.

SeongKu Kang, Yunyi Zhang, Pengcheng Jiang, Dongha Lee, Jiawei Han, and Hwanjo Yu. 2024. Taxonomy-guided semantic indexing for academic paper search. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 7169–7184. Association for Computational Linguistics.

Priyanka Kargupta, Nan Zhang, Yunyi Zhang, Rui Zhang, Prasenjit Mitra, and Jiawei Han. 2025. Taxoadapt: Aligning llm-based multidimensional taxonomy construction to evolving research corpora. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 29834–29850. Association for Computational Linguistics.

Avishek Lahiri, Yufang Hou, and Debarshi Kumar Sanyal. 2025. Taxoalign: Scholarly taxonomy generation using language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 30203–30223. Association for Computational Linguistics.

Carolyn E. Lipscomb. 2000. Medical subject headings (MeSH). *Bulletin of the Medical Library Association*, 88(3):265–266.

Sahil Mishra, Kumar Arjun, and Tanmoy Chakraborty. 2025. Rank, chunk and expand: Lineage-oriented reasoning for taxonomy expansion. In *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, volume ACL 2025 of *Findings of ACL*, pages 12935–12953. Association for Computational Linguistics.

Salvatore Raunich and Erhard Rahm. 2014. Target-driven merging of taxonomies with atom. *Inf. Syst.*, 42:1–14.

Stephen Roller, Douwe Kiela, and Maximilian Nickel. 2018. Hearst patterns revisited: Automatic hypernym detection from large text corpora. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, ACL 2018, Melbourne, Australia, July 15-20, 2018, Volume 2: Short Papers*, pages 358–363. Association for Computational Linguistics.

Jingbo Shang, Xinyang Zhang, Liyuan Liu, Sha Li, and Jiawei Han. 2020. Nettaxo: Automated topic taxonomy construction from text-rich network. In *WWW '20: The Web Conference 2020, Taipei, Taiwan, April 20-24, 2020*, pages 1908–1919. ACM / IW3C2.

Amanpreet Singh, Mike D'Arcy, Arman Cohan, Doug Downey, and Sergey Feldman. 2023. Scirepeval: A multi-format benchmark for scientific document representations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 5548–5566. Association for Computational Linguistics.

Rion Snow, Daniel Jurafsky, and Andrew Y. Ng. 2004. Learning syntactic patterns for automatic hypernym discovery. In *Advances in Neural Information Processing Systems 17 [Neural Information Processing Systems, NIPS 2004, December 13-18, 2004, Vancouver, British Columbia, Canada]*, pages 1297–1304.

Jiashuo Sun, Yixuan Xie, Jimeng Shi, Shaowen Wang, and Jiawei Han. 2026a. Tasr-rag: Taxonomy-guided structured reasoning for retrieval-augmented generation. *CoRR*, abs/2603.09341.

Yiqun Sun, Pengfei Wei, and Lawrence B. Hsieh. 2026b. Corpus2skill: Distilling enterprise knowledge into navigable agent skills for QA and RAG. In *Findings of the Association for Computational Linguistics: EMNLP 2026*. Association for Computational Linguistics.

Yanchao Tan, Carl Yang, Xiangyu Wei, Chaochao Chen, Longfei Li, and Xiaolin Zheng. 2022. Enhancing recommendation with automated tag taxonomy construction in hyperbolic space. In *38th IEEE International Conference on Data Engineering, ICDE 2022, Kuala Lumpur, Malaysia, May 9-12, 2022*, pages 1180–1192. IEEE.

Xuemei Tang, Chengxi Yan, Jinghang Gu, and Chu-Ren Huang. 2026. Chisagent: A multi-agent framework for event taxonomy construction in ancient chinese cultural systems. *CoRR*, abs/2601.05520.

Anthony K. H. Tung. 2026. DeepConnect: Harnessing AI to Elevate Collective Human Potential. AI as Shared Connective Infrastructure for Collective Sensemaking and Prudent Execution. SSRN preprint. Written 25 December 2025. Available at SSRN: `https://ssrn.com/abstract=6448879`.

Ulrike von Luxburg. 2007. A tutorial on spectral clustering. *Stat. Comput.*, 17(4):395–416.

Suyuchen Wang, Ruihui Zhao, Xi Chen, Yefeng Zheng, and Bang Liu. 2021. Enquire one's parent and child before decision: Fully exploit hierarchical structure for self-supervised taxonomy expansion. In *WWW '21: The Web Conference 2021, Virtual Event / Ljubljana, Slovenia, April 19-23, 2021*, pages 3291–3304. ACM / IW3C2.

Fei Xia, Yixuan Weng, Shizhu He, Kang Liu, and Jun Zhao. 2024. Find parent then label children: A two-stage taxonomy completion method with pre-trained language model. *CoRR*, abs/2406.17739.

Yue Yu, Yinghao Li, Jiaming Shen, Hao Feng, Jimeng Sun, and Chao Zhang. 2020. STEAM: self-supervised taxonomy expansion with mini-paths. In *KDD '20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, Virtual Event, CA, USA, August 23-27, 2020*, pages 1026–1035. ACM.

Qingkai Zeng, Yuyang Bai, Zhaoxuan Tan, Shangbin Feng, Zhenwen Liang, Zhihan Zhang, and Meng Jiang. 2024. Chain-of-layer: Iteratively prompting large language models for taxonomy induction from limited examples. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, CIKM 2024, Boise, ID, USA, October 21-25, 2024*, pages 3093–3102. ACM.

Chao Zhang, Fangbo Tao, Xiusi Chen, Jiaming Shen, Meng Jiang, Brian M. Sadler, Michelle Vanni, and Jiawei Han. 2018. Taxogen: Unsupervised topic taxonomy construction by adaptive term embedding and clustering. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD 2018, London, UK, August 19-23, 2018*, pages 2701–2709. ACM.

Kun Zhu, Lizi Liao, Yuxuan Gu, Lei Huang, Xiaocheng Feng, and Bing Qin. 2025. Context-aware hierarchical taxonomy generation for scientific papers via llm-guided multi-aspect clustering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 15616–15634. Association for Computational Linguistics.

A Appendix

A.1 Data Preparation

Tree Conversion. CCS is originally structured as a DAG, where certain concepts appear under multiple parent categories. We convert it to a strict tree via single-parent pruning, retaining only concepts with unambiguous single-parent assignments. This yields 1,768 concepts from the original taxonomy. Google Product Taxonomy is natively tree-structured and requires no conversion.

Sampling Protocol. For each target scale, we sample a connected rooted sub-taxonomy by random growth from the original taxonomy root: at each step, we randomly select a node whose parent has already been included in the sampled taxonomy, and then add its children as future expansion candidates. This preserves valid parent–child and ancestor relations inherited from the full taxonomy. To prevent leakage, few-shot examples are sampled from a separate top-level subtree pool, while test subsets are sampled from outside this pool. We evaluate CCS at up to 1K concepts, Google at up to 4K, SemEval-Food at 1K, and MeSH at 1K and 10K. All processed datasets are publicly available at `https://github.com/rebeccazyr/SPARROW`.

A.2 Implementation Details.

Model and Prompting Configuration. All methods use a 5-shot prompting setting. For TaxoGPT, Chain-of-Layer, and SPARROW, Table 1 reports means over five independent runs, with standard deviations in Table 8; Figure 4 instead reports one run per scale. LLMscorer outputs are deterministically decycled before evaluation. We use `GPT-5` via API with default decoding parameters, and deploy LLaMA3-8B locally on NVIDIA H100 GPUs with a temperature of 0.6.

LLMscorer Implementation. The original LMScorer scoring paradigm instantiates a natural-language template for each child–parent pair and ranks candidates using likelihood, pseudo-likelihood, or perplexity from a conventional language model. Exhaustively applying this formulation requires scoring all concept pairs, and perplexity is not a natural interface for modern chat models. We therefore implement LLMscorer in two stages. First, concept names are encoded with `allenai/specter2` (Singh et al., 2023), and the 50 most similar candidate parents for each child are retrieved by cosine similarity. Second, the child and its candidate list are presented to the evaluated chat backbone, which selects one parent. The resulting edges satisfy the single-parent constraint by construction, and cycles are removed deterministically before evaluation.

Spectral Clustering Configuration. For block construction, we apply non-overlapping spectral clustering over entity embeddings produced by `allenai/specter2`. The number of clusters K is selected automatically via a spectral eigengap heuristic. Specifically, for an entity set of size n, we define a backbone-dependent reference count $k^* = \lceil n/s_m \rceil$, where $s_m = 70$ for LLaMA3-8B-Instruct and $s_m = 100$ for other evaluated backbones. More generally, s_m can be chosen according to the backbone model's scale and reasoning capability. We then search over the candidate range $[\max(2, \lfloor 0.7k^* \rfloor), \min(\lceil 1.3k^* \rceil, n-1)]$, selecting the K that yields the largest eigengap. The affinity matrix is constructed from cosine distances as $\exp(-\gamma d^2)$ with $\gamma = 1.0$, using a nearest-neighbor graph with $n_neighbors = \min(n-1, 10)$.

Taxonomy Fusion Configuration. We initialize the global taxonomy using the largest block-level taxonomy and incrementally merge the remaining blocks in descending order of block size. Within each incoming block, nodes are processed in breadth-first order. Parent candidates are retrieved via cosine similarity over already-placed nodes in the `allenai/specter2` embedding space, retaining the top-10 candidates by default. Fusion proceeds in three steps: candidate scoping, parent selection, and sibling-versus-parent resolution.

Dataset	n	Method	Node	Edge	Anc.
SemEval-Food	1K	TaxoGPT	0.810	**0.606**	0.567
		Chain-of-Layer	0.513	0.315	0.227
		SPARROW	**0.965**	0.475	**0.595**
MeSH	1K	TaxoGPT	0.857	**0.547**	0.658
		Chain-of-Layer	0.271	0.113	0.166
		SPARROW	**0.964**	0.540	**0.700**
	10K	TaxoGPT	0.102	0.030	0.011
		Chain-of-Layer	0.040	0.010	0.015
		SPARROW	**0.902**	**0.360**	**0.487**

Table 6: Cross-domain and extreme-scale F1 results with GPT-5. Bold marks the best value for each dataset and scale.

Backbone	Method	Node	Edge	Anc.
LLaMA3.3-70B	TaxoGPT	0.042	0.015	0.018
	Chain-of-Layer	0.093	0.034	0.052
	SPARROW	**0.499**	**0.132**	**0.231**
gpt-oss-120b	TaxoGPT	0.268	0.064	0.115
	Chain-of-Layer	0.292	0.003	0.092
	SPARROW	**0.682**	**0.161**	**0.312**

Table 7: F1 results on CCS at the 1K scale with additional open-weight backbones.

A.3 Additional Experimental Results

Table 6 extends the GPT-5 evaluation beyond the two primary benchmarks. On SemEval-Food, SPARROW achieves the best Node and Ancestor F1, while TaxoGPT obtains the best Edge F1. On MeSH, SPARROW achieves the best Ancestor F1 at both 1K and 10K concepts and retains substantially stronger global structure at 10K. Table 7 shows the same relative advantage with two additional open-weight backbones on CCS. Table 8 reports variability across the five GPT-5 runs used in the main table.

A.4 Robustness and Block-Size Sensitivity

To isolate robustness to partitioning errors, we introduce controlled cut edges and compare the fused result with the corresponding unfused block taxonomies. Table 9 shows that fusion preserves a positive Ancestor F1 gain even at a 50% Cut Edge Ratio (CER), indicating that the merge stage can recover global paths despite substantial fragmentation. Table 10 varies the reference block size on CCS at 1K. Smaller blocks increase the number of cut edges and fusion tokens but also enlarge the recoverable Ancestor F1 gain; size 100 provides the best balance in our setting.

Dataset	Method	Node	Edge	Anc.
CCS	TaxoGPT	**0.024**	0.104	0.062
	Chain-of-Layer	0.285	0.077	0.117
	SPARROW	**0.024**	**0.032**	**0.035**
Google	TaxoGPT	0.031	0.120	0.082
	Chain-of-Layer	0.339	0.194	0.185
	SPARROW	**0.024**	**0.045**	**0.045**

Table 8: Standard deviations of GPT-5 F1 results over five runs. Bold marks the lowest deviation.

CER (%)	ΔNode F1	ΔEdge F1	ΔAnc. R	ΔAnc. F1
11.6	0.000	−0.011	+0.190	+0.148
30.0	0.000	+0.007	+0.135	+0.095
50.0	0.000	+0.017	+0.135	+0.088
Avg.	0.000	+0.004	+0.153	+0.110

Table 9: Gain from constraint-guided fusion over unfused block taxonomies under controlled partitioning noise. CER denotes Cut Edge Ratio.

A.5 Embedding-Space Connectivity

We compare gold subtrees with size-matched random concept sets in the `allenai/specter2` embedding space. Connectivity is the fraction of nodes belonging to the largest connected component of the induced kNN graph, and centroid purity measures concentration around the set centroid. Table 11 shows that gold subtrees are highly connected but only moderately centroid-compact. This supports graph-based partitioning while avoiding the stronger assumption that each subtree forms a compact convex cluster.

Block size	# blocks	IER	Node F1			Edge F1			Ancestor F1			Fusion tokens
			Local	Fused	Δ	Local	Fused	Δ	Local	Fused	Δ	
1000	1	1.000	0.855	0.855	0.000	0.278	0.278	0.000	0.397	0.397	0.000	0
500	2	0.845	0.940	0.940	0.000	0.365	0.367	+0.002	0.620	0.640	+0.020	25,862
100	10	0.766	0.969	0.969	0.000	0.411	0.412	+0.001	0.465	0.623	+0.158	52,058
50	20	0.587	0.997	0.997	0.000	0.369	0.379	+0.010	0.319	0.524	+0.205	60,471

Table 10: Block-size sensitivity on CCS at 1K scale. Δ is the F1 change from the local block taxonomies to the fused taxonomy; size 100 balances local induction, edge retention, and fusion cost.

Dataset	Connectivity	Centroid purity
CCS	90.8 / 13.9	41.0 / 3.3
Google	92.2 / 14.0	50.8 / 3.9

Table 11: Embedding-space structure of gold subtrees versus size-matched random sets (gold/random, %). Connectivity is measured in the SPECTER2 kNN graph; centroid purity measures concentration around a shared center.

A.6 Qualitative Fusion Cases

Recovering fragmentation. The local edge between *Data structures design and analysis* and its parent *Design and analysis of algorithms* is cut across blocks. Constraint-guided fusion retrieves the latter as the parent and restores the missing cross-block relation.

Correcting parent displacement. For the path *Computer vision* $\rightarrow$ *Computer vision tasks* $\rightarrow$ *Scene understanding*, treating the query as a sibling would flatten the hierarchy. Sibling-or-parent resolution instead places it below the selected node, preserving the intermediate level.

Unrecovered spurious parent. For *Spam detection*, whose gold parent is *Web search engines*, the system selects *Software and application security*. This semantically plausible but structurally incorrect attachment scopes the concept to the wrong subtree and illustrates the remaining sensitivity to candidate retrieval and local semantic ambiguity.

A.7 Prompts for SPARROW

Block-Level Taxonomy Induction

Input:

Concept set $\mathcal{C}$: a flat list of concepts.

Few-shot examples: concept sets paired with reference parent–child relations.

==========

Instruction:

Generate parent–child relations that organize the input concepts into a taxonomy hierarchy.

==========

Output:

A list of parent–child relations in the form:

`child is a subtopic of parent`

Parent Selection

Input:

Query concept Q (concept label, child concepts, and local taxonomy path hint)

Candidate parent set $\mathcal{P}$ (candidate labels, current child concepts and existing taxonomy paths)

==========

Decision criteria:

1. Subsumption: P is valid only if every child of Q can be semantically placed under P.

2. Granularity alignment: Among candidates passing subsumption, prefer P whose existing children are at a similar or slightly broader abstraction level than Q's children.

3. Depth continuity: Prefer attachments that preserve smooth hierarchical progression and avoid skipping semantic levels.

Fallback: If no non-root candidate meet requirements, select ROOT.

==========

Output:

The label of the selected parent, or ROOT

Sibling-or-Parent Resolution

Input:

Query concept Q (concept label, child concepts, and local taxonomy path hint)

selected parent P (candidate labels, current child concepts and existing taxonomy paths)

==========

Decision options:

Sibling: Q is attached as a direct child of P, if Q's scope is comparable in granularity to the existing children of P.

Parent: Q is inserted as an intermediate node between P and a subset of its existing children, if Q semantically subsumes that subset. The affected children are explicitly identified and moved under Q.

==========

Output:

The relationship type (sibling or parent), and if parent, the list of affected children to be moved under Q.